

critical-section-MOM

Eduard Toni Alexandru

September 2026

1 Analysis

The objective of this assignment is to design and implement a high-level middleware that provides distributed mutual exclusion (critical sections) for processes running in a distributed system.

The primary constraints are:

- **Transparency:** Processes must be able to use the lock without any knowledge of the network topology, process count, or the identity of other competitors.
- **Message-Oriented Middleware (MOM):** The coordination must be achieved exclusively via message passing through a MOM (RabbitMQ).
- **Fault Tolerance:** The system must not deadlock if a process crashes while competing for or holding the lock.

2 Design

The chosen architecture is based on the **Single-Token Passing** pattern.

2.1 Conceptual Model

A dedicated RabbitMQ queue serves as the lock manager. The “lock” itself is a physical message (the token) seeded into this queue. Because a message in RabbitMQ can only be routed to and consumed by exactly one worker at a time, mutual exclusion is mathematically guaranteed by the broker.

2.2 State Transitions and Fault Tolerance

When a process requests the lock, it consumes the token but *does not acknowledge it* (`autoAck = false`).

- **While holding the lock:** The token is kept in an *unacknowledged* state within RabbitMQ. It is invisible to other processes, forcing them to block.

- **Upon normal completion:** The process explicitly rejects the message and instructs RabbitMQ to requeue it, passing the lock to the next waiting process.
- **Upon process crash:** If a process dies inside the critical section, its TCP connection to RabbitMQ drops. The broker immediately detects the severed connection and automatically transitions the unacknowledged token back into the queue. This built-in mechanic prevents distributed deadlocks without requiring timeouts, leases, or heartbeat implementations.

3 Theoretical Guarantees

A robust distributed lock must satisfy three classical concurrency properties: Mutual Exclusion, Freedom from Deadlock, and Freedom from Starvation. This middleware leverages the inherent mechanics of RabbitMQ to fulfill all three mathematically.

3.1 Mutual Exclusion

Mutual exclusion is guaranteed through a Single-Token cardinality constraint combined with the broker’s atomic delivery guarantees. The `LockInitializer` explicitly seeds a single message (the token) into the queue. RabbitMQ’s routing and delivery protocol ensures that a single message is only dispatched to exactly one active consumer at any given time. Consequently, it is impossible for two distributed processes to hold the token (and thus enter the critical section) concurrently.

3.2 Freedom from Deadlock

In distributed architectures, deadlocks primarily occur when a node acquires a lock and subsequently experiences a fatal failure (e.g., kernel panic, network partition) before releasing it, permanently halting the system.

This implementation achieves freedom from deadlock by entirely avoiding time-based leases. Instead, it utilizes RabbitMQ’s `autoAck = false` mechanism. When a process acquires the lock, the token is not deleted; it is held in an “unacknowledged” state. If the client process crashes, its underlying TCP socket disconnects. RabbitMQ immediately detects this connection loss and automatically transitions the unacknowledged token back into the queue. This architectural trait ensures the system is self-healing.

3.3 Freedom from Starvation

Starvation occurs if a competing process is continually bypassed, preventing it from ever acquiring the lock. This is mitigated by RabbitMQ’s standard queue behavior, which strictly enforces a FIFO (First-In, First-Out) message delivery ordering.

Because the middleware utilizes a push-based subscription model with `basicQos(1)`, RabbitMQ tracks the waiting consumers. As the token is repeatedly returned to the queue via `basicReject(requeue=true)`, the broker delegates it to the next consumer in the FIFO line, guaranteeing absolute fairness and ensuring no process waits indefinitely.

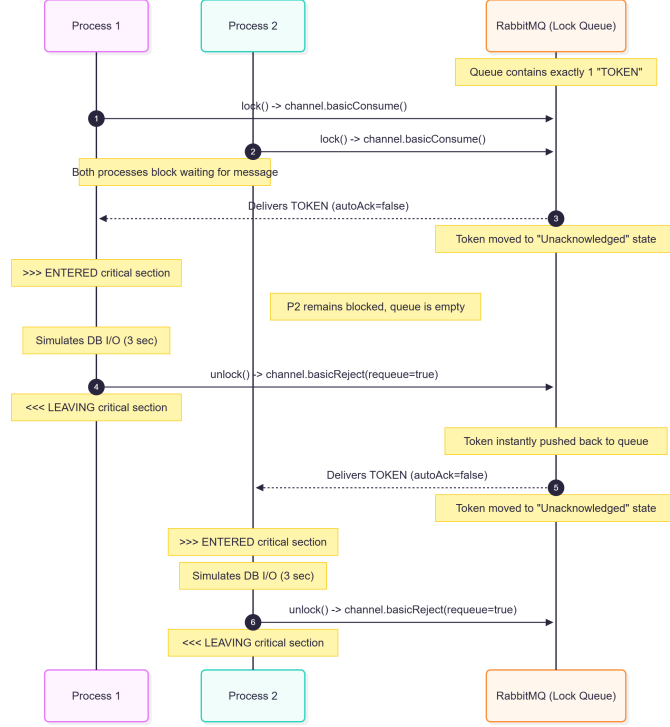


Figure 1: Flow diagram of the interactions between processes and RabbitMQ 1/2.

4 Implementation

The middleware is encapsulated within a `DistributedLock` class implementing `AutoCloseable` to ensure network resources are cleanly released.

4.1 Push-Based Acquisition

Initially, lock acquisition was implemented using a polling loop (`basicGet` combined with `Thread.sleep`). This was optimized to use `basicConsume`, turning it into a push-based model. A `CompletableFuture` is utilized to block the calling thread efficiently until RabbitMQ pushes the token over the network.

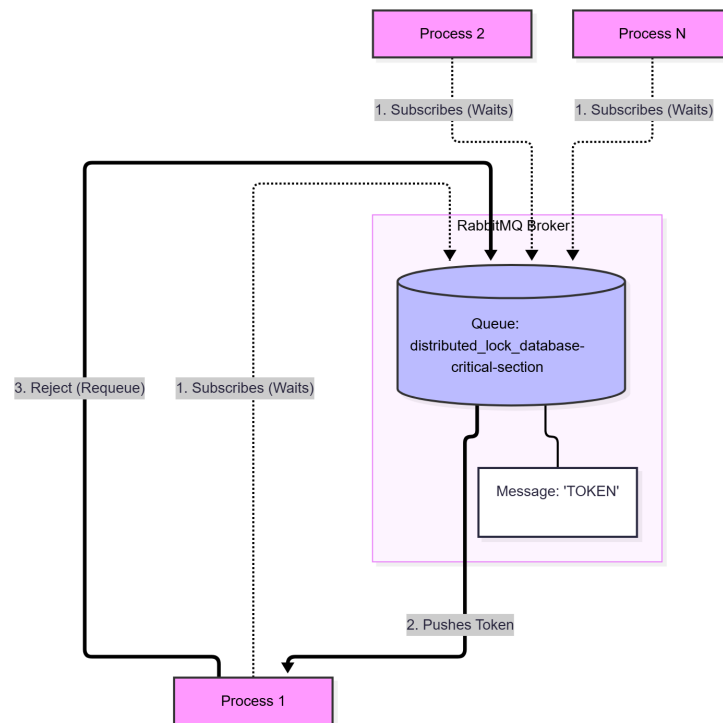


Figure 2: Flow diagram of the interactions between processes and RabbitMQ 2/2.

Furthermore, `basicQos(1)` is set on the channel to prevent the middleware from preemptively buffering the token before `lock()` is invoked.

4.2 Atomic Release

To release the lock, the middleware uses `channel.basicReject(lastDelivery, true)`. This is a critical implementation detail. An alternative approach (acknowledging the old token and publishing a new one) introduces a race condition: if the process crashes between the acknowledgment and the publication, the token is lost. `basicReject` with the requeue flag set to `true` ensures the lock is released atomically.

5 Implementation Details: Threading and Efficiency

The initial naive approach to lock acquisition involved an active polling loop using `basicGet` paired with `Thread.sleep()`. This approach is fundamentally flawed in a Message-Oriented Middleware context for two reasons:

1. **CPU Starvation:** Continuous polling wastes computational resources.
2. **Artificial Latency:** If the sleep interval is 1000ms, a process may wait up to 999ms to enter the critical section after the lock has been freed.

To resolve this, the optimized `DistributedLock` utilizes `channel.basicConsume` combined with Java's `CompletableFuture`. This transitions the architecture from a pull-based to a push-based model. The calling thread blocks natively at the JVM level, consuming zero CPU cycles. The moment the broker routes the token to the worker, the callback triggers `tokenFuture.complete()`, instantly resuming execution with sub-millisecond precision.

6 Testing

Testing distributed mutual exclusion requires simulating concurrent, independent processes.

To achieve this accurately, an `AutomatedDistributedTest` suite was developed. This suite uses Java's `ProcessBuilder` to programmatically spawn three entirely separate JVM instances.

1. A `LockInitializer` script runs first, using `queuePurge()` to remove orphaned tokens and `basicPublish()` to seed exactly one token.
2. The test suite launches the three independent JVMs simultaneously.
3. Standard output stream inheritance (`inheritIO()`) is used to interleave the logs of all processes into a single console.

Results: The console output strictly demonstrates mutual exclusion. Every `ENTERED critical section` log from a specific Process ID is strictly followed by a `LEAVING critical section` log from that same ID before any other process is granted entry. The system recovers elegantly if a process is forcefully terminated mid-execution, satisfying all requirements for the maximum grade constraint.